SALUS SECURITY

OCT 2026



# CODE SECURITY ASSESSMENT

GROUPMINE

# Overview

## Project Summary

- Name: Groupmine
- Platform: EVM-compatible chains
- Language: Solidity
- Repository:
    - https://github.com/tapeminecoop/groupmine
- Audit Range: See Appendix - 1

# Project Dashboard

## Application Summary

| | |
|---|---|
| **Name** | Tapeout-Groupmine |
| **Version** | v1 |
| **Type** | Solidity |
| **Dates** | Oct 28 2026 |
| **Logs** | Oct 28 2026 |

## Vulnerability Summary

| | |
|---|---|
| **Total High-Severity issues** | 0 |
| **Total Medium-Severity issues** | 3 |
| **Total Low-Severity issues** | 3 |
| **Total informational issues** | 2 |
| **Total** | 8 |

## Contact

E-mail: support@salusec.io



SALUS

# Risk Level Description

| | |
|---|---|
| **High Risk** | The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users. |
| **Medium Risk** | The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact. |
| **Low Risk** | The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances. |
| **Informational** | The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth. |



SALUS

# Content


**Introduction** **4**

1.1 About SALUS 4

1.2 Audit Breakdown 4

1.3 Disclaimer 4

**Findings** **5**

2.1 Summary of Findings 5

2.2 Notable Findings 6

1. Rewards locked when all circuits are revoked before registration 6
2. Admission ordering allows depositors to influence reward allocation 7
3. Centralization risk 8
4. Unbounded wallet revocation can prevent allocation corrections 9
5. Ownership transfer can prevent correction of over-issuance 10
6. Unsolicited receipt transfers can delay allocation finalization 11

2.3 Informational Findings 12

7. Missing events for functions that change critical state 12
8. Magic numbers are used 13

**Appendix** **14**

Appendix 1 - Files in Scope 14




SALUS

# Introduction

## 1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

## 1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

## 1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.



SALUS

# Findings

## 2.1 Summary of Findings

| ID | Title | Severity | Category | Status |
|---|---|---|---|---|
| 1 | Rewards locked when all circuits are revoked before registration | Medium | Business Logic | Pending |
| 2 | Admission ordering allows depositors to influence reward allocation | Medium | Business Logic | Pending |
| 3 | Centralization risk | Medium | Centralization | Pending |
| 4 | Unbounded wallet revocation can prevent allocation corrections | Low | Business Logic | Pending |
| 5 | Ownership transfer can prevent correction of over-issuance | Low | Business Logic | Pending |
| 6 | Unsolicited receipt transfers can delay allocation finalization | Low | Business Logic | Pending |
| 7 | Missing events for functions that change critical state | Informational | Logging | Pending |
| 8 | Magic numbers are used | Informational | Code Quality | Pending |



SALUS

## 2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

### 1. Rewards locked when all circuits are revoked before registration

| Severity: Medium | Category: Business Logic |
| --- | --- |
| Target:<br>- src/PoolMath.sol | |

### Description

All circuits created from the contributed NAND/LATCH are transferred directly to the Pool and revoked before any circuit is registered. Revocation settles their accrued BEM to the Pool, but revoked circuits cannot be registered. Consequently, the aggregate burn remains zero and `_sync()` cannot recognize the received rewards.

If all contributed materials have been consumed and every circuit has been revoked, no eligible circuit remains to establish the allocation ratio, permanently locking the received BEM.

src/PoolMath.sol:L79-L87

```
function accrue(uint256 total, State memory s) internal pure returns (Accrual memory a)
{
    uint256 weight = s.totalNandBurn + s.totalLatchBurn;
    if (weight != 0) {
        // delta value for participants.
        uint256 d = a.participants - s.participantBase;
        a.nandIncome += d * s.totalNandBurn / weight;
        a.latchIncome += d * s.totalLatchBurn / weight;
    }
}
```

### Recommendation

Provide a restricted settlement mechanism for the case where all circuits are revoked before initial registration, using verified burn data to determine reward allocation.



SALUS

## 2. Admission ordering allows depositors to influence reward allocation

| Severity: Medium | Category: Business Logic |
| --- | --- |
| Target:<br>- src/PoolMath.sol | |

## Description

Each admission checkpoints existing income using the previous composition, then applies the updated composition to incoming circuits' carried rewards. Depositors control admission order and batching, allowing the same set of circuits to produce different NAND/LATCH reward allocations.

For example, circuit A consumes 10 NAND and no LATCH, while circuit B consumes 5 NAND and 5 LATCH. Each carries 102 BEM of unclaimed rewards. Ignoring additional rewards between transactions, after A/B fees:

- Admit A first: its 100 participant BEM goes entirely to NAND. B's 100 is split using the combined 15:5 composition. Total: **175 NAND / 25 LATCH**.
- Admit B first: its 100 is split equally. A's 100 is then split using 15:5. Total: **125 NAND / 75 LATCH**.

A depositor can favor one receipt category by choosing the admission sequence, shifting rewards away from the other category.

src/PoolMath.sol:L79-L87

```
function accrue(uint256 total, State memory s) internal pure returns (Accrual memory a)
{
    uint256 weight = s.totalNandBurn + s.totalLatchBurn;
    if (weight != 0) {
        // delta value for participants.
        uint256 d = a.participants - s.participantBase;
        a.nandIncome += d * s.totalNandBurn / weight;
        a.latchIncome += d * s.totalLatchBurn / weight;
    }
}
```

## Recommendation

Commit to the round's intended NAND/LATCH allocation ratio before collecting rewards, or establish an atomic admission process for the complete initial circuit set.



SALUS

## 3. Centralization risk

| Severity: Medium | Category: Centralization |
| --- | --- |

Target:
- src/TapeoutEntitlementReceipt.sol
- src/TapeoutReceiptDistributor.sol

## Description

In the TapeoutEntitlementReceipt/TapeoutReceiptDistributor etc contracts, there exists some privileged roles, for example `owner`. These roles have the authority to execute some key functions such as `issue`, `allocate`, etc.

If these roles' private keys are compromised, an attacker could trigger these functions to harm the whole protocol.

src/TapeoutEntitlementReceipt.sol:L60-L77

```
function issue(address to, uint256 count) external onlyOwner returns (uint256 firstId) {
    // The owner cannot issue any new ERC1155 after the finalization.
    if (issuanceFinalized) revert IssuanceClosed();
    if (count == 0 || count > MAX_ISSUE_BATCH) revert InvalidCount();
    firstId = _supply + 1;
    uint256[] memory ids = new uint256[](count);
    uint256[] memory values = new uint256[](count);
    for (uint256 i; i < count; ++i) {
        ids[i] = firstId + i;
        values[i] = 1;
    }
    _mintBatch(to, ids, values, "");
    emit Issued(to, firstId, count, _supply);
}
```

## Recommendation

We recommend transferring ownership to a multi-signature account protected by a timelock. This ensures that no single party can unilaterally redirect treasury flows and gives users advance notice of treasury changes.



SALUS

## 4. Unbounded wallet revocation can prevent allocation corrections

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- src/TapeoutReceiptDistributor.sol | |

## Description

The function `revokeWallet()` iterates over every range and receipt ID allocated to a wallet. Allocations can accumulate across multiple transactions, while revocation must complete in a single transaction.

Large allocations may exceed the transaction gas limit during revocation, preventing the owner from correcting erroneous beneficiaries.

src/TapeoutReceiptDistributor.sol:L159-L180

```
function revokeWallet(address wallet) external onlyOwner {
    if (allocationFinalized) revert AllocationClosed();
    Range[] storage list = _ranges[wallet];
    uint256[2] memory removed;
    for (uint256 r; r < list.length; ++r) {
        Range storage range = list[r];
        mapping(uint256 => address) storage beneficiary = _beneficiary[range.category];
        for (uint256 i; i < range.count; ++i) {
            delete beneficiary[uint256(range.firstId) + i];
        }
        removed[uint256(range.category)] += range.count;
    }
    // remove allocated.
    _allocated[0] -= removed[0];
    _allocated[1] -= removed[1];
    delete _ranges[wallet];
    emit WalletRevoked(wallet, removed[0], removed[1]);
}
```

## Recommendation

Support bounded, incremental revocation of ranges and receipt IDs across multiple transactions.



SALUS

## 5. Ownership transfer can prevent correction of over-issuance

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- src/TapeoutEntitlementReceipt.sol | |

## Description

The function `retract()` requires the caller to be both the current owner and the holder of the highest receipt IDs. Transferring ownership does not transfer receipts held by the previous owner.

The new owner cannot independently correct previous over-issuance, even before issuance finalization. Correction requires cooperation from the previous holder.

src/TapeoutEntitlementReceipt.sol:L81-L98

```
function retract(uint256 count) external onlyOwner {
    // owners cannot retract if this is finalized.
    if (issuanceFinalized) revert IssuanceClosed();
    if (count == 0 || count > _supply || count > MAX_ISSUE_BATCH) revert InvalidCount();
    uint256 firstId = _supply - count + 1;
    uint256[] memory ids = new uint256[](count);
    uint256[] memory values = new uint256[](count);
    for (uint256 i; i < count; ++i) {
        ids[i] = _supply - i; // highest first, so the remaining IDs stay 1..S
        values[i] = 1;

        if (_holders[ids[i]] != msg.sender) revert NotIssuerHeld(ids[i]);
    }
    // burn batch.
    _burnBatch(msg.sender, ids, values);
    emit Retracted(firstId, count, _supply);
}
```

## Recommendation

Complete issuance corrections before ownership transfer, or transfer the relevant highest-ID receipts to the new owner during the handover.



SALUS

## 6. Unsolicited receipt transfers can delay allocation finalization

| Severity: Low | Category: Business Logic |
| --- | --- |
| Target:<br>- src/TapeoutReceiptDistributor.sol | |

## Description

Before finalization, any holder can transfer eligible receipts into the Distributor, increasing `_inventory` without increasing `_allocated`. Finalization requires both values to match.

A holder can front-run finalization with an unsolicited transfer, causing it to revert until the owner allocates the additional receipts. Repeated transfers can cause further delays and transaction costs.

src/TapeoutReceiptDistributor.sol:L185-L201

```
function finalizeAllocation() external onlyOwner {
    if (allocationFinalized) revert AllocationClosed();
    if (!nandReceipt.issuanceFinalized() || !latchReceipt.issuanceFinalized()) {
        revert IssuanceNotFinalized();
    }
    // c --> category.
    for (uint256 c; c < 2; ++c) {
        if (_allocated[c] != _inventory[c]) {
           revert InventoryNotCovered(TapeoutMinerPool.Category(c), _allocated[c],
_inventory[c]);
        }
    }
    allocationFinalized = true;
    emit AllocationFinalized(_inventory[0], _inventory[1], _wallets.length);
}
```

## Recommendation

Introduce a funding closure stage before allocation finalization, or restrict receipt funding to authorized senders.



SALUS

# 2.3 Informational Findings

### 7. Missing events for functions that change critical state

| Severity: Informational | Category: Logging |
| --- | --- |
| Target:<br>- src/canary/CanaryParticipant.sol | |

## Description

Events allow capturing the changed parameters so that off-chain tools/interfaces can register such changes that allow users to evaluate them. Missing events do not promote transparency and if such changes immediately affect users' perception of fairness or trustworthiness, they could exit the protocol causing a reduction in protocol users.

src/canary/CanaryParticipant.sol:L40-L50

```
function bindPool(RecoverableTapeoutMinerPool candidate) external onlyController {
    if (address(pool) != address(0) || address(candidate).code.length == 0) revert
InvalidBinding();
    if (
        candidate.recoveryController() != controller || candidate.recipientA() !=
controller
            || candidate.recipientB() != address(this) || address(candidate.bem()) !=
address(bem)
            || address(candidate.nandReceipt()) != nandReceipt
            || address(candidate.latchReceipt()) != latchReceipt
    ) revert InvalidBinding();
    pool = candidate;
}
```

## Recommendation

It is recommended to emit events for critical state changes.



SALUS

## 8. Magic numbers are used

| Severity: Informational | Category: Code Quality |
| --- | --- |
| Target:<br>- src/canary/RecoverableTapeoutMinerPool.sol | |

## Description

There are a few occurrences of literal values being used. To improve the code's readability and facilitate refactoring, consider defining a constant for every magic number, giving it a clear and self-explanatory name.

src/canary/RecoverableTapeoutMinerPool.sol:L571-L586

```
function emergencyReturn(uint256 id) external nonReentrant {
    if (msg.sender != recoveryController) revert NotRecoveryController();
    // close at first, then return.
    if (!closed) revert CanaryOpen();
    if (id != 3204 && id != 12485) revert UnauthorizedCanaryMiner(id);
    address holder = minerCollection.ownerOf(id);
    // Idempotent after an interrupted runner; never touches a miner held by somebody
else.
    if (holder == recoveryController) return;
    if (holder != address(this)) revert MinerNotHeld(id, holder);
    // transfer nft back to the recovery controller.
    minerCollection.transferFrom(address(this), recoveryController, id);
    if (minerCollection.ownerOf(id) != recoveryController) revert MinerNotHeld(id,
address(this));
    returned[id] = true;
    emit CanaryMinerReturned(id, recoveryController, _miners[id].registered);
   }
```

## Recommendation

Consider defining a constant variable for the magic number.



SALUS

# Appendix

## Appendix 1 - Files in Scope

This audit covered the following files in commit c3cb933:

| File | SHA-1 hash |
|---|---|
| CanaryParticipant.sol | a105b64b498f66331dfab2f3a16a5d773b4494d1 |
| RecoverableTapeoutMinerPool.sol | 68f236c354df6a7670629bbf908532e8462d0917 |
| TapeoutMinerPool.sol | 0c42cad5f48e8ef421ecf86fa9b3936fbe78afc0 |
| TapeoutEntitlementReceipt.sol | 7a69466d7fc5bb7e68bff92887bfb6fcc6435fea |
| TapeoutReceiptDistributor.sol | 7d54d14560ff63374a8c2deecc6e8e02d4e13f84 |
| PoolMath.sol | 41e288d8a90da2f9f596d281a5d4eb2b0d749e58 |



SALUS